VELOSALESAI

MASTER BLUEPRINT

Product vision • Architecture • AI safety • UX • Roadmap • Testing

A practical plan for building a trustworthy Business AI for Nigerian businesses

Prepared as a working reference for the founder and coding agent. This document describes intended product behavior; it does not certify that the current codebase already implements these capabilities.

Version 1.0 • 13 October 2026

How to use this blueprint

Keep the PDF as the readable master plan. Save the companion VELOSALESAI_BUILD_SPEC.md in the repository so Codex can read and follow the project requirements. Verify the current codebase before treating any planned capability as implemented.

Immediate priority: Mobbin-guided UI/UX redesign. The blueprint is broad by design; implementation must remain staged.

1. Executive summary

VeloSalesAI is a business assistant for Nigerian small businesses and growing companies. Its long-term ambition is to help a business understand its own operations and assist with sales, customer communication, orders, and decision-making. It should be more than a generic chatbot: its answers and actions must be grounded in the business's actual records, policies, and permissions.

The immediate priority is a premium, practical UI/UX redesign guided by the user's Mobbin references. After that, the product should establish one reliable end-to-end sales workflow before adding advanced automation.

Product principle: make the first workflow reliable and useful before expanding the feature list. Never claim a business outcome that has not been measured.

2. Vision, mission, and positioning

Vision

A trusted Business AI that understands a company's products, customers, orders, policies, and performance—and helps the owner run the business with appropriate human control.

Mission

Help businesses respond to enquiries accurately, turn suitable enquiries into well-recorded orders, and make better decisions using their own data.

Initial positioning

Start with Nigerian small businesses and growing sellers. Make the product useful to both groups over time, while validating one initial customer segment before attempting to serve every industry.

Potential value proposition

VeloSalesAI helps businesses manage customer enquiries and sales with an assistant grounded in their real business information, with clear handoff to people and measurable results.

3. Target users and validation

Initial customer hypotheses

- Small social-commerce sellers who receive repeated questions about products, prices, stock, delivery, and payment.
- Growing online businesses that need consistent responses, order tracking, staff handoff, and useful reporting.
- Businesses with enough recurring enquiries for response speed and follow-up to matter.

Validate before overbuilding

- Interview 10–15 business owners in a chosen segment.
- Ask to see their current enquiry, catalogue, and order workflow—with permission.

- Document common questions, lost-sale causes, order mistakes, channel preferences, and willingness to pay.
- Recruit approximately five pilot businesses if possible; obtain permission before using real data.

These are validation targets, not evidence that market demand or sales improvement has already been proven.

4. Product architecture: two experiences, one trusted data foundation

A. Customer-facing sales assistant

- Answer approved questions about products, prices, stock, delivery, payment methods, and business policies.
- Recommend products using actual catalogue information and the customer's stated needs.
- Collect required order details and clearly explain what has and has not been submitted or confirmed.
- Escalate complaints, uncertainty, sensitive requests, and unsupported actions to a human.

B. Owner-facing Business AI

- Answer questions about the owner's own business using authorized records.
- Summarize enquiries, orders, catalogue performance, and sales trends where reliable data exists.
- Explain the records behind an insight and state limitations or missing data.
- Suggest next steps and execute only actions allowed by backend permissions and confirmation rules.

Shared foundation

Both experiences may use shared business data, but must have different roles and permissions. Customer-facing conversations must never expose private owner analytics, internal notes, secrets, or another customer's records.

5. Core sales workflow

The first complete business workflow should be measurable from enquiry to order and human handoff. Implement the smallest dependable version before advanced features.

1. A customer sends an enquiry through the first supported channel.
2. The backend identifies the correct business and loads permitted catalogue and policy context.
3. The assistant answers from verified data; it admits uncertainty if required information is missing.
4. If the customer wants to buy, the assistant collects only the necessary details.
5. The backend validates product availability, quantities, prices, delivery information, and required fields.
6. The system creates an order record with an explicit initial status.
7. The customer and owner receive accurate status information; the owner can review or correct details.
8. Uncertain, exceptional, or disputed cases are handed to a human.

An AI statement such as “order confirmed” is not proof that an order exists. Confirmation must reflect the actual backend state.

6. Eight strategic capability areas

1. Business AI Brain

An owner-facing assistant that answers questions using verified business records, explains which records support an answer, surfaces uncertainty, and proposes permitted actions.

2. Lost-Sale Rescue

Identify stalled or unanswered enquiries where follow-up is appropriate. Suggest a follow-up and track its outcome. Automated outreach must respect customer consent, channel rules, rate limits, and owner approval settings.

3. Demand Radar

Find recurring requests for unavailable products and patterns in enquiry, conversion, and stock data. Initially present signals and evidence; do not claim reliable forecasting without sufficient history.

4. Nigerian Sales Intelligence

Investigate support for Nigerian English, Pidgin, local delivery patterns, bargaining policies, and common payment workflows. Validate with real users; do not stereotype customers or assume one workflow fits every state or business.

5. Unified Customer Conversation History

Where integrations, permissions, platform policies, and reliable identity matching allow, provide a consolidated view of customer conversations. Do not merge identities based on weak guesses.

6. Repeat-Customer Engine

Use verified purchase history and customer preferences to support service and relevant offers. Apply consent, privacy, opt-out, and frequency controls.

7. Natural-Language Workflow Builder

Let an owner describe a workflow in ordinary language. Convert it into a visible, reviewable set of triggers, conditions, and actions. Require approval for consequential actions and provide run history, error handling, and a pause control.

8. Reliable Sales Engine

The foundation connecting catalogue, enquiries, order capture, order statuses, customer handoff, and outcome measurement. Treat this as a core product capability, not an optional add-on.

7. UI/UX principles and screen priorities

Design direction: premium SaaS, clear hierarchy, restrained decoration, fast scanning, and excellent mobile responsiveness. Use Mobbin references supplied by the user as inspiration for layout and interaction patterns; do not copy proprietary assets or assume reference screenshots reveal backend behavior.

Priority screens

1. Main dashboard.
2. AI conversations.
3. Product catalogue.
4. Analytics and sales.
5. Settings and integrations.
6. Mobile experience across the above screens.

Required UI states

- Loading, empty, error, offline or unavailable, permission denied, and success states.
- Accessible contrast, readable type, clear focus/keyboard behavior where relevant, and consistent spacing.
- Tables, filters, search, pagination, and clear status labels for operational data.
- Confirmation dialogs for destructive or consequential actions.
- Mobile layouts that preserve the primary task rather than merely shrinking desktop screens.

UI redesign is Phase 1. Do not add new channels or large feature sets as part of the redesign unless required to preserve existing behavior.

8. Technical architecture and existing stack

Known stack from project context (must be verified against the repository before changing anything): Vercel for frontend hosting, Supabase for database/authentication, Render for an API/server, and one or more AI providers previously explored. Do not assume every listed provider is currently integrated or production-ready.

Architecture responsibilities

- Frontend: present data and collect user intent; do not hold private server secrets.
- Backend/API: authorize requests, enforce business rules, validate data, call providers, and perform controlled actions.
- Database: maintain authoritative records, constraints, and tenant ownership.
- AI abstraction: standardize model calls and provider errors so providers can be changed with limited application impact.
- Background jobs: use only where asynchronous tasks, retries, schedules, or webhook processing require them.
- Observability: capture structured errors, latency, token usage, and important action events without unnecessarily logging sensitive content.

Architecture rules

- Inspect repository, package files, environment variable names, routes, database schema, and current flows before implementation.
- Preserve existing working functionality and patterns unless a documented reason supports a change.
- Do not expose API keys or service-role credentials in client code or version control.
- Do not add infrastructure or paid dependencies without explaining the need, alternatives, cost, and approval request.
- Treat provider free tiers as changeable; establish budgets, rate limits, and fallback behavior.

9. Data model and tenancy

The exact schema must be based on the current application. The following is a conceptual checklist, not an instruction to create duplicate tables blindly.

- Businesses/tenants: business identity, settings, plan, and usage limits.
- Memberships and roles: which authenticated users can access each business and which actions they may perform.
- Products and variants: identifiers, descriptions, prices, availability, stock and images.
- Customers: minimum necessary profile, consent/preferences, and verified purchase relationships.
- Conversations and messages: channel, participants, timestamps, status, and handoff information.
- Orders and order items: price snapshots, quantities, delivery details, payment state, fulfilment state, and audit history.

- Integration connections: provider, authorized scopes, token storage, health, and disconnect/revocation state.
- Usage events and audit logs: model usage, important actions, failures, and actor attribution.

Tenant isolation

Every tenant-owned record must be scoped and authorized. Use Supabase Row Level Security where appropriate, server-side authorization, foreign keys and constraints, and tests proving that one business cannot access another business's data. Never trust a tenant ID supplied by the browser without verifying membership and permissions.

10. AI reliability and action safety

- Ground answers in current, authorized records; distinguish retrieved facts from suggestions.
- Use structured tool schemas and server-side validation for actions.
- Treat user messages, product descriptions, uploaded documents, and retrieved text as untrusted content that cannot override system policies.
- Do not let the model decide access permissions; enforce permissions in application code.
- Use explicit confidence/uncertainty behavior and human escalation for missing or conflicting data.
- Prevent duplicate side effects with idempotency keys and safe retry handling.
- Use timeouts, bounded retries, rate limits, circuit breakers or fallback responses as appropriate.
- Require explicit approval for refunds, bulk outreach, irreversible changes, and other consequential actions.
- Maintain an evaluation set of real-world-style test questions and expected outcomes before changing prompts or models.

11. Privacy, security, and compliance

- Collect only data required for the product and document why it is needed.
- Use secure authentication, least privilege, tenant isolation, and server-side authorization.
- Keep secrets server-side and rotate exposed credentials promptly.
- Protect webhooks with provider signature verification and replay/duplicate handling where supported.
- Define retention, deletion, export, backup, and incident-response procedures.
- Review AI provider retention and processing terms before sending personal or confidential business data.
- Support consent and opt-out for customer communications and respect messaging platform rules.
- Publish clear terms and privacy information and investigate applicable Nigerian data-protection requirements, including the Nigeria Data Protection Act where applicable.

This blueprint is a product/engineering plan, not legal advice. Obtain qualified advice when required, particularly before handling sensitive data, payments, or high-volume messaging.

12. AI provider strategy

Groq, Gemini, OpenAI, Claude, SambaNova, and routing services have been explored previously; current integrations, access and terms must be verified. Avoid hard-coding business logic to a single model's unique output behavior.

- Benchmark providers on accuracy, structured tool use, latency, uptime, context limits, cost, rate limits, and data handling.
- Use inexpensive models for low-risk classification and simple tasks when evaluation shows adequate quality.
- Use stronger models for complex reasoning only where needed.

- Keep deterministic validation and business rules outside the model.
- Track token and request usage by tenant; enforce plan limits and budgets.
- Create safe behavior for provider failure, rate limiting, malformed output, and missing context.

Do not promise customers a permanently free AI service until usage economics and provider terms have been verified.

13. Cost and monetization

Cost categories to measure: model input/output, hosting, database/storage, messaging, image/file storage, background jobs, observability, payment fees, support and taxes where applicable.

Measure direct service cost per active business and per conversation. Use observed usage—not guesses—to set quotas and price tiers.

- A free tier may help onboarding, but needs abuse controls and clear limits.
- Paid tiers may vary by conversation volume, team size, channels, analytics and automation; validate willingness to pay.
- Do not rely on advertising revenue as the only funding source until the model is validated.
- Show plan limits, renewal terms, and usage clearly.

The founder has limited resources; prefer existing infrastructure and free development tools where practical. Avoid unnecessary paid services and subscriptions.

14. Integrations and channels

Treat each integration as a separate delivery with provider-specific approval, access, webhook, consent, pricing, and reliability requirements.

- Start with the channel most important to the initial pilot customers.
- Verify the current official API and policy for WhatsApp, Instagram, Telegram, payment, email, and any future commerce provider before implementation.
- Do not assume access to personal accounts, message history, or automation that the provider does not permit.
- Use secure token storage, explicit connect/disconnect flows, status visibility, and webhook verification.
- Keep integration failure visible and provide a safe manual fallback.

15. Development phases and acceptance criteria

Phase 0 — Repository audit

- Inventory current pages, routes, components, APIs, data schema, authentication, integrations, tests, and deployment configuration.
- Report what works, what is broken, what is incomplete, and what risks exist. Do not rewrite the project during the audit.

Phase 1 — UI/UX redesign (first priority)

- Use the supplied Mobbin references and existing functionality to create a screen-by-screen plan.
- Implement the agreed screens incrementally and preserve behavior.
- Verify desktop and mobile layouts, loading/empty/error states, accessibility basics, and existing flows.

Phase 2 — Reliable sales engine

- Prove the complete enquiry-to-order workflow with correct state transitions, duplicate-event handling, handoff, and tests.

Phase 3 — Business AI Brain

- Answer owner questions from authorized data; show relevant evidence; handle missing data; prevent cross-tenant leakage; validate proposed actions.

Phase 4 — Advanced capabilities

- Prioritize Lost-Sale Rescue, Demand Radar, Nigerian Sales Intelligence, unified conversations, repeat-customer tools, and workflow builder using pilot evidence.

Definition of done for each task

- Clear acceptance criteria are met.
- Relevant tests and build/lint checks pass, or failures are reported honestly.
- No known regression to existing core behavior.
- Security and tenant isolation are considered.
- The implementation summary lists changed files, tests run, known limitations, and any required manual setup.

16. Testing and product metrics

Required test scenarios

- Accurate product price and stock response; missing or stale product data.
- Out-of-stock product and invalid quantity.
- Policy conflict, discount request, complaint, or human handoff.
- Order creation, status transitions, cancellation and duplicate webhook.
- Two businesses attempting to access each other's data.
- AI provider timeout, rate limit, malformed response, and recovery.
- Usage limit reached and background job retry.
- Customer consent/opt-out and deletion request.

Pilot metrics

- Answer accuracy and rate of unsupported claims.
- Enquiry response time and percentage handled appropriately.
- Enquiry-to-order conversion, using a defined denominator and time window.
- Order correction/cancellation/error rate.
- Human handoff rate and reasons.
- Active businesses, repeat usage and retention.
- Cost per conversation and direct cost per business.

Establish a baseline and report sample size and time period. Do not equate conversations with sales or claim causation without evidence.

17. Risks and mitigations

- Feature sprawl: keep the first release focused on one customer segment and one complete workflow.
- Hallucinated business facts: ground answers in current records and refuse to guess.
- Cross-tenant data leakage: enforce RLS/server authorization and test adversarial access attempts.
- Uncontrolled costs: per-tenant quotas, model routing, alerts and budgets.
- Messaging restrictions: verify official APIs, consent requirements and policy before rollout.
- Provider outage: timeouts, retries, graceful failure and manual fallback.
- Poor onboarding: guided setup, sample data, clear validation and owner confirmation.
- Unproven market demand: conduct interviews and pilots before major infrastructure or marketing spend.

18. Decisions, assumptions, and open questions

Confirmed from project context

- Product name: VeloSalesAI; website: velosalesai.com.ng; app domain: app.velosalesai.com.ng.
- Vision: a business AI that understands and helps run the entire business.
- Target: small sellers and growing businesses, with appropriate capabilities for each.
- Initial market: Nigerian businesses.
- UI redesign based on Mobbin references is the first priority.
- Priority screens: dashboard, AI conversations, catalogue, analytics/sales, settings/integrations, mobile experience.
- Known stack to verify: Vercel, Supabase and Render; AI provider options explored but not all confirmed as integrated.

Assumptions to verify

- Which messaging channel is most important to initial pilot customers.
- Current repository state and existing implementation of each workflow.
- Actual model and hosting costs under expected usage.
- The initial customer segment and willingness to pay.

Open decisions

- Which segment should be the first pilot target?
- Which one messaging channel will be supported first?
- Which existing features are complete, partially working, or only planned?
- Which model providers have valid keys and production access?
- What is the current deployment and test status?

Do not let Codex guess these answers. It should inspect the repository where possible and ask concise questions when a decision materially affects implementation.

19. Codex operating instructions

Use this section as the high-level operating contract for the coding agent. Pair it with the Markdown build specification in the project repository.

First inspect the repository and report architecture, working functionality, risks and a proposed plan. Do not modify files during the initial audit.

Treat the current codebase as the source of truth for what is implemented; treat this blueprint as product intent, not proof that features already exist.

The first implementation task is the UI/UX redesign using user-supplied Mobbin references. Do not start new integrations or advanced AI capabilities before the UI task is agreed.

Make small, reversible changes. Preserve existing working behavior and avoid wholesale rewrites.

Do not add paid services, dependencies, migrations, or destructive changes without explaining the reason and requesting approval where appropriate.

Never expose secrets, weaken authentication, disable security policies, or bypass tenant isolation to make a demo work.

Run relevant tests and checks. Report exactly what passed, failed, was not run, and why.

At task completion, summarize files changed, user-visible behavior, data/schema changes, tests, known issues, and next step.

Codex must not claim that a feature works merely because a UI was created. Trace the flow through frontend, backend, database and provider integration.

20. First next actions

Collect Mobbin screenshots for the dashboard and highest-priority screens; share them with the coding agent.

Save this PDF and the companion Markdown specification in the VeloSalesAI project folder/repository.

Open the project in Codex and request a read-only repository audit using the included instructions.

Review the audit and agree the UI redesign scope before code changes.

Implement and test the UI redesign in small steps.

After the redesign, select one pilot business workflow and prove it end to end.

Recruit pilot users, measure the baseline, and use evidence to decide what to build next.